

PolyFlop8 ALU

Unit Test Report

Comprehensive Verification of Arithmetic Logic Unit

Document ID:	TC-ALU-001
Revision:	1.0
Date:	January 5, 2026
Test Level:	Unit/Block Level
Test Engineer:	Artem Horiunov
Status:	PASS

1 TEST OVERVIEW

1.1 Unit Under Test (UUT)

Component	Arithmetic Logic Unit (ALU)
VHDL Entity	ALU
Source File	ALU_tb.vhd ALU.vhd
Test Environment	Vivado v2025.1 (64-bit) SW Build: 6140274 on Thu May 22 00:12:29 MDT 2025 IP Build: 6138677 on Thu May 22 03:10:11 MDT 2025 SharedData Build: 6139179 on Tue May 20 17:58:58 MDT 2025
Clock Frequency	100MHz (10ns period)
Time Resolution	1ns

1.2 Scope

This test report documents the verification of the PolyFlop8 ALU component. The ALU performs all arithmetic and logical operations, generates status flags, and selects operands via internal multiplexers.

1.3 Test Objectives

- Verify all arithmetic operations produce correct 8-bit results
- Validate flag generation logic (H, V, N, C, Z)
- Confirm proper multiplexer selection for operand sources
- Test 2's complement handling and overflow detection
- Ensure 100% code coverage of ALU behavioral description

2 REQUIREMENTS COVERAGE

Req ID	Requirement Description	Test Case ID	Status
TR-ALU-01	Verify all arithmetic/logical operations produce correct 8-bit results	TC-ALU-001-01	PASS
TR-ALU-02	Verify flags (H, V, N, C, Z) are correctly asserted for each operation	TC-ALU-001-02	PASS
TR-ALU-03	Verify alu_mux_sel_a correctly selects between data sources	TC-ALU-001-03	PASS
TR-ALU-04	Verify alu_mux_sel_b correctly selects between data sources	TC-ALU-001-04	PASS
TR-ALU-05	Verify 2's complement handling in NEG and SUB operations	TC-ALU-001-05	PASS

Table 1: Requirements Coverage Matrix

3 DETAILED TEST CASES

3.1 Test Case TC-ALU-001-01: Basic Arithmetic Operations

Requirement: TR-ALU-01

Purpose: Verify all arithmetic/logical operations produce correct 8-bit results.

Status: PASS

3.2 Test Case TC-ALU-001-02: Flag Generation

Requirement: TR-ALU-02

Purpose: Verify flags (H, V, N, C, Z) are correctly asserted.

Status: PASS

3.3 Test Case TC-ALU-001-03: MUX A Selection

Requirement: TR-ALU-03

Purpose: Verify alu_mux_sel_a selects correctly.

Status: PASS

3.4 Test Case TC-ALU-001-04: MUX B Selection

Requirement: TR-ALU-04

Purpose: Verify alu_mux_sel_b selects correctly.

Status: PASS

3.5 Test Case TC-ALU-001-05: 2's Complement Operations

Requirement: TR-ALU-05

Purpose: Verify 2's complement handling.

Status: PASS

4 SOURCE CODES

4.1 ALU.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ALU is
    Port (
        pc_data          : in  STD_LOGIC_VECTOR (7 downto 0);
        regfile_data_a    : in  STD_LOGIC_VECTOR (7 downto 0);
        rs_imm_in         : in  STD_LOGIC_VECTOR (7 downto 0);
        regfile_data_b    : in  STD_LOGIC_VECTOR (7 downto 0);
        alu_mux_sel_a     : in  STD_LOGIC;
        alu_mux_sel_b     : in  STD_LOGIC;
        alu_sel           : in  STD_LOGIC_VECTOR (3 downto 0);
        cin               : in  STD_LOGIC;
        result            : out STD_LOGIC_VECTOR (7 downto 0);
        flags             : out STD_LOGIC_VECTOR (4 downto 0)
    );
end ALU;

architecture Behavioral of ALU is
    signal op_a : STD_LOGIC_VECTOR (7 downto 0);
```

```

signal op_b : STD_LOGIC_VECTOR (7 downto 0);

constant ALU_ADD : std_logic_vector(3 downto 0) := "0000";
constant ALU_ADC : std_logic_vector(3 downto 0) := "0001";
constant ALU_SUB : std_logic_vector(3 downto 0) := "0010";
constant ALU_SBC : std_logic_vector(3 downto 0) := "0011";
constant ALU_AND : std_logic_vector(3 downto 0) := "0100";
constant ALU_OR  : std_logic_vector(3 downto 0) := "0101";
constant ALU_XOR : std_logic_vector(3 downto 0) := "0110";
constant ALU_MOV : std_logic_vector(3 downto 0) := "0111";
constant ALU_LDI : std_logic_vector(3 downto 0) := "1000";
constant ALU_COM : std_logic_vector(3 downto 0) := "1001";
constant ALU_NEG : std_logic_vector(3 downto 0) := "1010";

begin
  op_a <= pc_data when alu_mux_sel_a = '1' else regfile_data_a;
  op_b <= rs_imm_in when alu_mux_sel_b = '1' else regfile_data_b;

  process(op_a, op_b, alu_sel, cin)
    variable v_op_a : unsigned(8 downto 0);
    variable v_op_b : unsigned(8 downto 0);
    variable v_result : unsigned(8 downto 0);
    variable v_cin : unsigned(0 downto 0);
    variable f_z, f_c, f_n, f_v, f_h : std_logic;
  begin
    v_op_a := resize(unsigned(op_a), 9);
    v_op_b := resize(unsigned(op_b), 9);
    v_cin(0) := cin;

    case alu_sel is
      when ALU_ADD =>
        v_result := v_op_a + v_op_b;
        f_h := (op_a(3) and op_b(3)) or (op_b(3) and not v_result(3)) or
          ↪ (op_a(3) and not v_result(3));
        f_v := (op_a(7) and op_b(7) and not v_result(7)) or (not op_a(7)
          ↪ and not op_b(7) and v_result(7));
        f_c := v_result(8);
      when ALU_ADC =>
        v_result := v_op_a + v_op_b + v_cin;
        f_h := (op_a(3) and op_b(3)) or (op_b(3) and not v_result(3)) or
          ↪ (op_a(3) and not v_result(3));
        f_v := (op_a(7) and op_b(7) and not v_result(7)) or (not op_a(7)
          ↪ and not op_b(7) and v_result(7));
        f_c := v_result(8);
      when ALU_SUB =>
        v_result := v_op_a - v_op_b;
        f_v := (op_a(7) and not op_b(7) and not v_result(7)) or (not
          ↪ op_a(7) and op_b(7) and v_result(7));
        f_c := v_result(8);
      when ALU_SBC =>
        v_result := v_op_a - v_op_b - v_cin;
        f_v := (op_a(7) and not op_b(7) and not v_result(7)) or (not
          ↪ op_a(7) and op_b(7) and v_result(7));
        f_c := v_result(8);
      when ALU_AND =>
        v_result := v_op_a and v_op_b;
      when ALU_OR =>
        v_result := v_op_a or v_op_b;
      when ALU_XOR =>
        v_result := v_op_a xor v_op_b;
      when ALU_COM =>
        v_result := not v_op_a;
        f_c := '1';
    end case;
  end process;

```

```

        when ALU_NEG =>
            v_result := 0 - v_op_a;
            if v_result(7 downto 0) = "10000000" then f_v := '1'; else f_v
                ↪ := '0'; end if;
            if v_result(7 downto 0) /= "00000000" then f_c := '1'; else f_c
                ↪ := '0'; end if;
        when ALU_MOV =>
            v_result := v_op_b;
        when ALU_LDI =>
            v_result := v_op_a;
        when others =>
            v_result := (others => '0');
    end case;

    if (v_result(7 downto 0) = "00000000") then
        f_z := '1';
    else
        f_z := '0';
    end if;
    f_n := v_result(7);

    result <= std_logic_vector(v_result(7 downto 0));
    flags <= f_h & f_v & f_n & f_c & f_z;
end process;
end Behavioral;

```

4.2 ALU_tb.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ALU_tb is
end ALU_tb;

architecture Behavioral of ALU_tb is
    component ALU
        Port (
            pc_data          : in  STD_LOGIC_VECTOR (7 downto 0);
            regfile_data_a   : in  STD_LOGIC_VECTOR (7 downto 0);
            rs_imm_in        : in  STD_LOGIC_VECTOR (7 downto 0);
            regfile_data_b   : in  STD_LOGIC_VECTOR (7 downto 0);
            alu_mux_sel_a    : in  STD_LOGIC;
            alu_mux_sel_b    : in  STD_LOGIC;
            alu_sel           : in  STD_LOGIC_VECTOR (3 downto 0);
            cin              : in  STD_LOGIC;
            result            : out STD_LOGIC_VECTOR (7 downto 0);
            flags             : out STD_LOGIC_VECTOR (4 downto 0)
        );
    end component;

    signal pc_data          : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal regfile_data_a   : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal rs_imm_in        : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal regfile_data_b   : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal alu_mux_sel_a    : STD_LOGIC := '0';
    signal alu_mux_sel_b    : STD_LOGIC := '0';
    signal alu_sel          : STD_LOGIC_VECTOR (3 downto 0) := (others => '0');
    signal cin              : STD_LOGIC := '0';
    signal result           : STD_LOGIC_VECTOR (7 downto 0);
    signal flags            : STD_LOGIC_VECTOR (4 downto 0);

```

```

constant OP_ADD : std_logic_vector(3 downto 0) := "0000";
constant OP_ADC : std_logic_vector(3 downto 0) := "0001";
constant OP_SUB : std_logic_vector(3 downto 0) := "0010";
constant OP_SBC : std_logic_vector(3 downto 0) := "0011";
constant OP_AND : std_logic_vector(3 downto 0) := "0100";
constant OP_OR  : std_logic_vector(3 downto 0) := "0101";
constant OP_XOR : std_logic_vector(3 downto 0) := "0110";
constant OP_MOV : std_logic_vector(3 downto 0) := "0111";
constant OP_LDI : std_logic_vector(3 downto 0) := "1000";
constant OP_COM : std_logic_vector(3 downto 0) := "1001";
constant OP_NEG : std_logic_vector(3 downto 0) := "1010";

begin
  uut: ALU PORT MAP (
    pc_data => pc_data,
    regfile_data_a => regfile_data_a,
    rs_imm_in => rs_imm_in,
    regfile_data_b => regfile_data_b,
    alu_mux_sel_a => alu_mux_sel_a,
    alu_mux_sel_b => alu_mux_sel_b,
    alu_sel => alu_sel,
    cin => cin,
    result => result,
    flags => flags
  );

  stim_proc: process
    procedure check_alu(
      desc : string;
      exp_res : integer;
      exp_flags : std_logic_vector(4 downto 0)
    ) is
    begin
      wait for 10 ns;
      assert std_match(result, std_logic_vector(to_unsigned(exp_res, 8)))
        report desc & "Result Mismatch! Expected: " & integer'image(
          exp_res) &
          "Got: " & integer'image(to_integer(unsigned(result)))
          severity error;

      if (exp_flags(4) /= 'X') then assert flags(4) = exp_flags(4) report
        desc & "HFlagError" severity error; end if;
      if (exp_flags(3) /= 'X') then assert flags(3) = exp_flags(3) report
        desc & "VFlagError" severity error; end if;
      if (exp_flags(2) /= 'X') then assert flags(2) = exp_flags(2) report
        desc & "NFlagError" severity error; end if;
      if (exp_flags(1) /= 'X') then assert flags(1) = exp_flags(1) report
        desc & "CFlagError" severity error; end if;
      if (exp_flags(0) /= 'X') then assert flags(0) = exp_flags(0) report
        desc & "ZFlagError" severity error; end if;
    end procedure;

  begin
    report "Starting ALU Verification (TC-ALU-001)...";

    regfile_data_a <= x"10";
    pc_data <= x"20";
    regfile_data_b <= x"01";
    rs_imm_in <= x"02";

    alu_mux_sel_a <= '0'; alu_mux_sel_b <= '0';
    alu_sel <= OP_ADD;
    check_alu("MUX: RegA+RegB", 17, "00000");
  end
end

```

```

alu_mux_sel_a <= '1'; alu_mux_sel_b <= '0';
check_alu("MUX:␣PC␣+␣RegB", 33, "00000");

alu_mux_sel_a <= '0'; alu_mux_sel_b <= '1';
check_alu("MUX:␣RegA␣+␣Imm", 18, "00000");

alu_mux_sel_a <= '1'; alu_mux_sel_b <= '1';
check_alu("MUX:␣PC␣+␣Imm", 34, "00000");

alu_mux_sel_a <= '0';
alu_mux_sel_b <= '0';

regfile_data_a <= x"0A"; regfile_data_b <= x"14";
alu_sel <= OP_ADD;
check_alu("ADD:␣Basic", 30, "00000");

regfile_data_a <= x"64"; regfile_data_b <= x"64";
check_alu("ADD:␣Signed␣Overflow", 200, "01100");

regfile_data_a <= x"0F"; regfile_data_b <= x"01";
check_alu("ADD:␣Half␣Carry", 16, "10000");

cin <= '1';
regfile_data_a <= x"0A"; regfile_data_b <= x"0A";
alu_sel <= OP_ADC;
check_alu("ADC:␣With␣Carry", 21, "10000");

cin <= '0';
regfile_data_a <= x"FF"; regfile_data_b <= x"01";
check_alu("ADC:␣Carry␣Chain␣(0xFF+1)", 0, "10011");

regfile_data_a <= x"14"; regfile_data_b <= x"0A";
alu_sel <= OP_SUB;
check_alu("SUB:␣Basic", 10, "X0000");

regfile_data_a <= x"37"; regfile_data_b <= x"37";
check_alu("SUB:␣Zero␣Result", 0, "X0001");

cin <= '1';
regfile_data_a <= x"14"; regfile_data_b <= x"0A";
alu_sel <= OP_SBC;
check_alu("SBC:␣With␣Borrow", 9, "X0000");
cin <= '0';

regfile_data_a <= x"0F"; regfile_data_b <= x"55";
alu_sel <= OP_AND;
check_alu("AND", 5, "XX0XX");

alu_sel <= OP_OR;
check_alu("OR", 95, "XX0XX");

regfile_data_a <= x"55"; regfile_data_b <= x"FF";
alu_sel <= OP_XOR;
check_alu("XOR", 170, "XX1X0");

regfile_data_a <= x"AA"; regfile_data_b <= x"BB";
alu_sel <= OP_MOV;
check_alu("MOV␣(Pass␣B)", 187, "XXXXX");

alu_sel <= OP_LDI;
check_alu("LDI␣(Pass␣A)", 170, "XXXXX");

```

```
    regfile_data_a <= x"00";
    alu_sel <= OP_COM;
    check_alu("COM: Zero", 255, "XX110");

    regfile_data_a <= x"01";
    alu_sel <= OP_NEG;
    check_alu("NEG: -1", 255, "X0110");

    regfile_data_a <= x"80";
    check_alu("NEG: Overflow", 128, "X1110");

    report "ALU Verification Complete.";
    wait;
end process;
end Behavioral;
```

5 SIMULATION LOG

```
Time resolution is 1 ps
Note: Starting ALU Verification (TC-ALU-001)...
Time: 0 ps Iteration: 0 Process: /ALU_tb/stim_proc File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ALU_TB.vhd
Note: Testing Multiplexers...
Time: 0 ps Iteration: 0 Process: /ALU_tb/stim_proc File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ALU_TB.vhd
Note: Testing Arithmetic...
Time: 40 ns Iteration: 0 Process: /ALU_tb/stim_proc File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ALU_TB.vhd
Note: Testing Logic...
Time: 120 ns Iteration: 0 Process: /ALU_tb/stim_proc File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ALU_TB.vhd
Note: Testing Pass-Through & Unary...
Time: 150 ns Iteration: 0 Process: /ALU_tb/stim_proc File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ALU_TB.vhd
Note: ALU Verification Complete.
Time: 200 ns Iteration: 0 Process: /ALU_tb/stim_proc File: F:/Projects/
    ↪ PolyFlop8-MCU/Polyflop8/Polyflop8.srcs/sources_1/new/Unit_test_bench/
    ↪ ALU_TB.vhd
```

6 PASS/FAIL CRITERIA

6.1 Success Criteria

Criteria	Description
Functional	All test cases execute without assertion failures
Timing	No timing violations (ALU is combinational)
Coverage	100% statement coverage, 100% branch coverage achieved
Consistency	Identical results across multiple simulation runs
Documentation	All test results properly documented and signed off

6.2 Failure Conditions

- Any single assertion failure during simulation
- Code coverage less than 100% for ALU behavioral code
- Timing violations or simulation warnings
- Inconsistent results between simulation runs
- Missing or incomplete test documentation

6.3 Coverage Goals

- **Statement Coverage:** 100% of ALU process statements executed
- **Branch Coverage:** 100% of case statement branches taken
- **Condition Coverage:** All flag calculation conditions tested
- **Toggle Coverage:** All signal bits toggled 0→1 and 1→0
- **FSM Coverage:** N/A (ALU is combinational)

7 SPECIAL TEST CONSIDERATIONS

7.1 Edge Cases

- Test all arithmetic operations with 0x00 and 0xFF operands
- Verify proper handling of signed vs unsigned overflow
- Test NEG operation with minimum signed value (0x80)
- Verify carry chain behavior through all bit positions
- Test all possible multiplexer combinations

7.2 Timing Considerations

- ALU is purely combinational - no clock dependencies
- Monitor for glitches or hazards during input transitions
- Verify stable outputs within specified propagation delay
- Test with minimum, typical, and maximum delay models

7.3 Integration Considerations

- Verify interface compatibility with Register File
- Test handshake signals with Control Unit
- Validate flag output format matches Status Register expectations
- Confirm result bus loading and fanout capabilities